

CUIアプリケーション (main.py) 作成手順書 (修正版)

1. 概要

本ドキュメントは、CUI (Character User Interface) で動作するオセロゲームのメインプログラム `main.py` の実際のコードに基づいた作成手順を記述する。

このファイルは、`othello`パッケージの各クラス (`Game`, `Board`, `Player`など) を連携させ、ユーザーとの対話を通じてゲーム全体を進行させるエントリーポイントとしての役割を担う。

開発手法としては、UIとユーザー操作のフローが中心となるため、処理の流れを上から順に実装していく**トップダウン**のアプローチで解説する。すべての実装コードには**型ヒント**を付与する。

前提知識: `othello/game.py`の`Game`クラスはプレイヤーオブジェクトを管理せず、`current_player`として石の色(`Disc`)を保持する。そのため、`main.py`側でプレイヤーオブジェクトの管理と切り替えを行う必要がある。

2. 実装ステップ

ステップ1: 必要なモジュールとクラスのインポート

ゲームの進行に必要なクラスと、コンピュータの思考時間に遅延を入れるための`time`モジュールをインポートする。

```
import time
from othello.game import Game
from othello.disc import Disc
from othello.player.human_player import HumanPlayer
from othello.player.computer_player import ComputerPlayer
from othello.player.strategies import STRATEGIES # strategies.pyで定義された辞書
```

ステップ2: ゲーム設定の選択機能

ユーザーにゲームモード (対人/対CPU) と、CPU戦の場合の難易度を選択させ、対応する`Player`オブジェクトのペアを返すヘルパー関数を作成する。

- 関数名はアンダースコアで始まる `_select_game_settings` とし、このファイル内でのみ使用することを意図する。
- プレイヤー1は常に人間の黒 (先手)、プレイヤー2を対戦相手として設定する。

```
def _select_game_settings():
    """
    ユーザーにゲームモードとAIの難易度を選択させる
    """
    # モード選択
    while True:
        mode = input("Select game mode (1: Human vs Human, 2: Human vs Computer):")
```

```

    ")

    if mode in ["1", "2"]:
        break
    print("Invalid input. Please enter 1 or 2.")

player1 = HumanPlayer(Disc.BLACK)
player2 = None

if mode == "1":
    player2 = HumanPlayer(Disc.WHITE)
    print("\nStarting Human vs Human game.")
else:
    # AI難易度選択
    strategy_names = list(STRATEGIES.keys())
    while True:
        print("\nSelect computer difficulty:")
        for i, name in enumerate(strategy_names):
            print(f"{i + 1}: {name}")

        try:
            choice = int(input(f"Enter number (1-{len(strategy_names)}): "))
            if 1 <= choice <= len(strategy_names):
                strategy_name = strategy_names[choice - 1]
                strategy_class = STRATEGIES[strategy_name]
                player2 = ComputerPlayer(strategy_class(), Disc.WHITE)
                print(f"\nStarting Human vs Computer ({strategy_name}) game.")
                break
            else:
                print("Invalid number.")
        except ValueError:
            print("Please enter a valid number.")

    return player1, player2

```

ステップ3: メインのゲームループ実装

ゲームの初期化から終了までの主要な流れを管理する関数を作成する。

- この`game_loop`関数は、`Game`オブジェクトと、ステップ2で作成した2つの`Player`オブジェクトを受け取る。
- `main.py`側で現在のプレイヤーオブジェクト (`current_player`) を管理し、ターンごとに入れ替える。
- `game.is_game_over()`が`True`になるまでループを続ける。
- `print(game.board)`で盤面を表示する。(Boardクラスに`__str__`メソッドが実装されていることが前提)
- 人間のターンの場合は入力を受け付け、コンピュータのターンの場合は`make_move()`を呼び出す。
- `game.play_turn(x, y)`を呼び出すと、`Game`クラス内部で石の配置と手番の色の切り替えが行われる。

```

def game_loop(game, player1, player2):
    """
    CUIでのゲームのメインループ
    """
    current_player = player1

    while not game.is_game_over():
        print(game.board)
        player_name = "Black" if current_player.disc_color == Disc.BLACK else
"White"
        print(f"Current player: {player_name}")

        valid_moves = game.board.get_valid_moves(current_player.disc_color)
        if not valid_moves:
            print("No valid moves. Passing the turn.")
            game.switch_player() # Gameオブジェクトに色の切り替えを任せる
            current_player = player2 if current_player == player1 else player1
            continue

        if isinstance(current_player, HumanPlayer):
            print("Valid moves:", valid_moves)
            while True:
                try:
                    move_str = input("Enter your move (x,y): ")
                    x_str, y_str = move_str.split(',')
                    x, y = int(x_str), int(y_str)
                    if (x, y) in valid_moves:
                        game.play_turn(x, y)
                        break
                    else:
                        print("Invalid move. Please choose from the valid moves.")
                except ValueError:
                    print("Invalid input format. Please use 'x,y'.")
            else: # ComputerPlayer
                print("Computer is thinking...")
                time.sleep(1)
                move = current_player.make_move(game.board)
                print(f"Computer plays: {move}")
                game.play_turn(move[0], move[1])

        # main.py側でプレイヤーオブジェクトを切り替える
        current_player = player2 if current_player == player1 else player1

```

ステップ4: 結果表示機能

ゲーム終了後に最終的な盤面、スコア、勝者を表示する関数を作成する。

```

def show_result(game):
    """
    ゲームの結果を表示する
    """

```

```
print("\n--- Game Over ---")
print(game.board)
winner = game.get_winner()
if winner:
    winner_name = "Black" if winner == Disc.BLACK else "White"
    print(f"Winner: {winner_name}!")
else:
    print("It's a draw!")

black_discs, white_discs = game.board.count_discs()
print(f"Final score: Black {black_discs} - White {white_discs}")
```

ステップ5: 全体を統括するmain関数

これまで作成した関数を呼び出し、ゲーム全体の流れを制御する`main`関数を作成する。

```
def main():
    """
    CUIでオセロゲームを実行する
    """

    # 1. ゲーム設定
    player1, player2 = _select_game_settings()

    # 2. ゲームオブジェクトの初期化
    game = Game()

    # 3. メインループの実行
    game_loop(game, player1, player2)

    # 4. 結果表示
    show_result(game)
```

ステップ6: 実行ブロックの追加

このスクリプトが直接実行された場合に`main()`関数が呼び出されるように、お決まりの実行ブロックを追加して完成。

```
if __name__ == "__main__":
    main()
```

3. まとめ

以上で`main.py`が完成する。この手順書は、`Game`クラスの仕様を正しく理解した上で、CUIの進行管理をどのように実装するかを示している。

Written by ポリテクセンター栃木